

Name: Tanishk Dhope

Roll No: 27

Batch: T12

DevOps: Assignment 1

Aim: To understand DevOps: Principles, practices & DevOps engineer role & responsibilities.

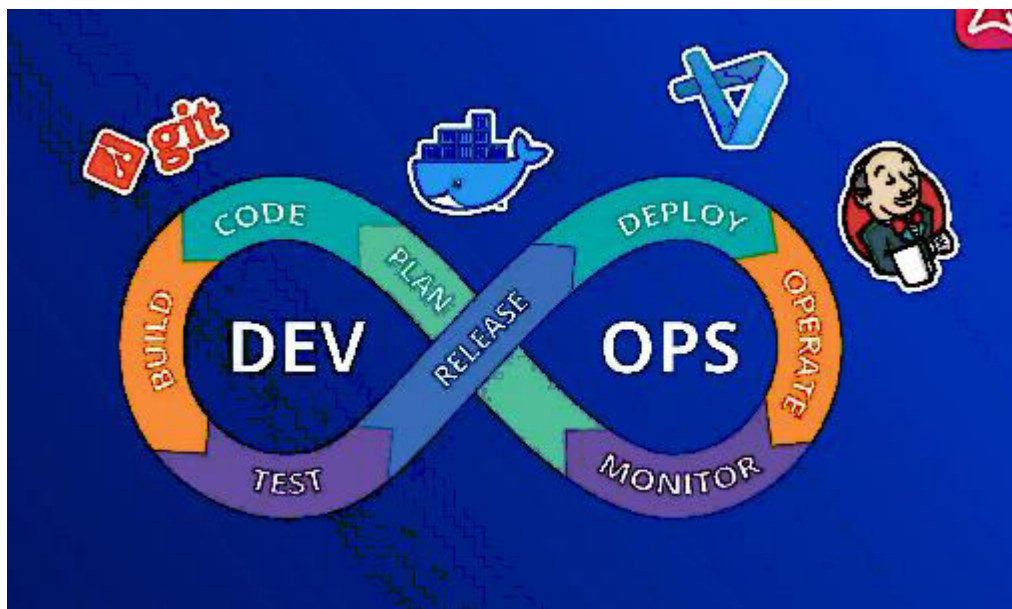
Theory:

DevOps is a set of practices, tools, and cultural philosophies that combine software development (Dev) and IT operations (Ops) to improve collaboration, automate processes, and accelerate the delivery of high-quality software.

The core principles of DevOps focus on fostering collaboration, automation, and continuous improvement to deliver high-quality software efficiently.

1. **Collaboration and Culture:** Encourage shared responsibility between development, operations, and other teams, breaking down silos to align goals and improve communication.
2. **Automation:** Automate repetitive tasks like testing, deployment, and infrastructure provisioning to increase efficiency, reduce errors, and speed up delivery.

3. **Continuous Integration and Continuous Delivery (CI/CD):** Regularly integrate code changes into a shared repository (CI) and automate deployment to production (CD) to ensure fast, reliable releases.
4. **Monitoring and Feedback:** Continuously monitor systems and applications, using real-time data and user feedback to identify issues, improve performance, and guide development.
5. **Incremental Improvement:** Deliver small, frequent updates to minimize risk, enable faster iterations, and adapt to changing requirements.
6. **Infrastructure as Code (IaC):** Manage infrastructure through code to ensure consistency, scalability, and repeatability in environment setup.
7. **Customer-Centric Approach:** Prioritize user needs and business value, aligning development and operations efforts to deliver meaningful outcomes.



DevOps Practices

DevOps practices are actionable approaches that implement the core principles of DevOps to streamline software development, deployment, and operations. Below are the key DevOps practices:

Continuous Integration (CI): Developers frequently merge code changes into a shared repository, with automated builds and tests to detect issues early. Tools like Jenkins, GitLab CI/CD, or CircleCI are commonly used.

Continuous Delivery (CD): Extends CI by automatically deploying code changes to staging or production-like environments after passing tests, ensuring reliable and frequent releases. Tools like Spinnaker or ArgoCD support this practice.

Continuous Deployment: Automatically deploys every code change that passes tests directly to production, enabling rapid delivery. This requires robust automation and testing (e.g., using AWS CodePipeline or GitHub Actions).

Infrastructure as Code (IaC): Manages infrastructure using code (e.g., Terraform, AWS CloudFormation) to provision and configure environments consistently and scalably.

Configuration Management: Automates and standardizes system configurations using tools like Ansible, Puppet, or Chef to ensure consistency across environments.

Automated Testing: Implements comprehensive test suites (unit,

integration, end-to-end) to validate code changes automatically, reducing manual effort and catching bugs early.

Continuous Monitoring and Observability: Tracks application and infrastructure performance in real-time using tools like Prometheus, Grafana, or Datadog to identify issues and optimize systems based on data-driven insights.

Version Control: Uses tools like Git to manage code and configuration changes, enabling collaboration, traceability, and rollback capabilities.

Containerization and Orchestration: Packages applications into containers (e.g., Docker) and manages them at scale with orchestration tools like Kubernetes for portability and scalability.

Collaboration and Communication: Fosters team collaboration through tools like Slack, Microsoft Teams, or Jira, ensuring alignment and transparency across development and operations.

Microservices Architecture: Designs applications as small, independent services that communicate via APIs, enabling faster development, deployment, and scaling.

Site Reliability Engineering (SRE): Applies software engineering practices to operations, focusing on reliability, scalability, and automation (e.g., defining Service Level Objectives with tools like New Relic).

Security Integration (DevSecOps): Embeds security practices into the DevOps pipeline, using tools like Snyk or OWASP ZAP to scan for vulnerabilities and ensure secure code.

Continuous Feedback and Improvement: Collects feedback from users and systems to iteratively improve processes, products, and team

workflows, often using retrospectives or post-mortems.

DevOps Engineer Role and Responsibilities

A DevOps Engineer plays a crucial role in bridging the gap between software development and IT operations. Their primary goal is to streamline the development lifecycle by implementing automation, continuous integration, and continuous delivery (CI/CD) practices.

Automation of Development & Deployment Pipelines:

CI/CD Implementation: Automating the build, test, and deployment pipelines using tools like Jenkins, GitLab CI, CircleCI, and Bamboo.

Scripting: Writing scripts to automate repetitive tasks, such as deployments and server provisioning, often using tools like Bash, Python, or Ruby.

Infrastructure as Code (IaC): Managing infrastructure using code (e.g., using Terraform, Ansible, Chef, Puppet) to ensure consistent, repeatable environments.

Collaboration and Communication:

Bridging Development & Operations: Working closely with software developers, QA engineers, and IT teams to ensure smooth transitions from development to production.

Agile Collaboration: Often participating in Agile or Scrum methodologies, including sprint planning and retrospectives, to ensure the development process aligns with operational needs.

Documentation: Creating and maintaining documentation for deployment procedures, systems configurations, and troubleshooting.

System Monitoring and Performance

Monitoring: Implementing tools to monitor applications and systems (e.g., Prometheus, Grafana, Nagios) to track performance, identify bottlenecks, and ensure uptime.

Alerting: Setting up automated alerts for system failures, performance degradation, or any unusual activity to proactively address issues.

Optimization: Tuning performance for applications, databases, or networks to ensure speed, reliability, and scalability.

Infrastructure Management

Cloud Management: Managing cloud infrastructure (AWS, Azure, Google Cloud) to ensure cost-effective, scalable, and secure environments.

Provisioning: Automating the provisioning of servers, databases, networking, and storage using IaC tools.

Containerization & Orchestration: Using Docker for containerization and Kubernetes for orchestration to manage scalable, resilient, and distributed applications.

Virtualization: Leveraging virtual machines (VMs) and virtualization tools for resource optimization.

Security and Compliance

Security Automation: Integrating security best practices into the CI/CD

pipeline, ensuring secure code deployment, patching, and vulnerability scanning.

Access Control: Implementing identity and access management (IAM) policies to ensure proper permissions across different environments and systems.

Compliance Auditing: Ensuring systems adhere to industry standards and regulations (e.g., GDPR, HIPAA) by implementing auditing and logging practices.

Troubleshooting & Incident Management

Root Cause Analysis: Investigating and troubleshooting issues in production and staging environments.

Incident Response: Responding to outages or performance issues, coordinating fixes, and implementing post-mortem analyses to avoid future occurrences.

Disaster Recovery: Creating and testing backup and recovery strategies to minimize downtime and data loss during outages.

DevOps Tools

Jenkins: An open-source automation server used for building, testing, and deploying code continuously.

Docker: A platform that enables developers to package applications and dependencies into lightweight containers for consistent environments.

Kubernetes: An open-source container orchestration tool for automating deployment, scaling, and management of containerized applications.

Git: A distributed version control system used to track code changes and collaborate on software development projects.

GitHub: A web-based platform for Git repositories, providing version control, collaboration features, and code review tools.

GitLab: A DevOps platform that integrates Git repositories with CI/CD pipelines, project management, and code quality tools.

Conclusion

In conclusion, DevOps tools are essential for automating, streamlining, and optimizing the software development lifecycle. By leveraging tools for CI/CD, configuration management, containerization, cloud infrastructure, and monitoring, DevOps engineers can enhance collaboration, improve system reliability, and accelerate delivery cycles. These tools enable continuous improvement, scalability, and efficiency across development and operations teams.

LO Mapping - LO1

Assignment 2

Aim: To Understand Version Control System, Git Installation and GitHub Account.

Theory:

What is Version Control System?

A Version Control System (VCS) is a software tool that helps developers manage and track changes to their code over time. It allows multiple people to work on the same project simultaneously without overwriting each other's work by keeping a detailed history of all modifications.

This history enables developers to review, compare, and restore previous versions of the code if needed.

Version control systems also support branching, which lets developers work on new features or fixes independently before merging their changes back into the main project.

What is git?

Git is a popular distributed version control system designed to help developers track changes in their code, collaborate efficiently, and manage different versions of a project.

Unlike centralized systems, Git allows every user to have a complete copy of the entire project history on their own computer, which makes working offline and merging changes easier and faster.

It supports features like branching and merging, enabling developers to work on separate tasks or features independently before combining their work.

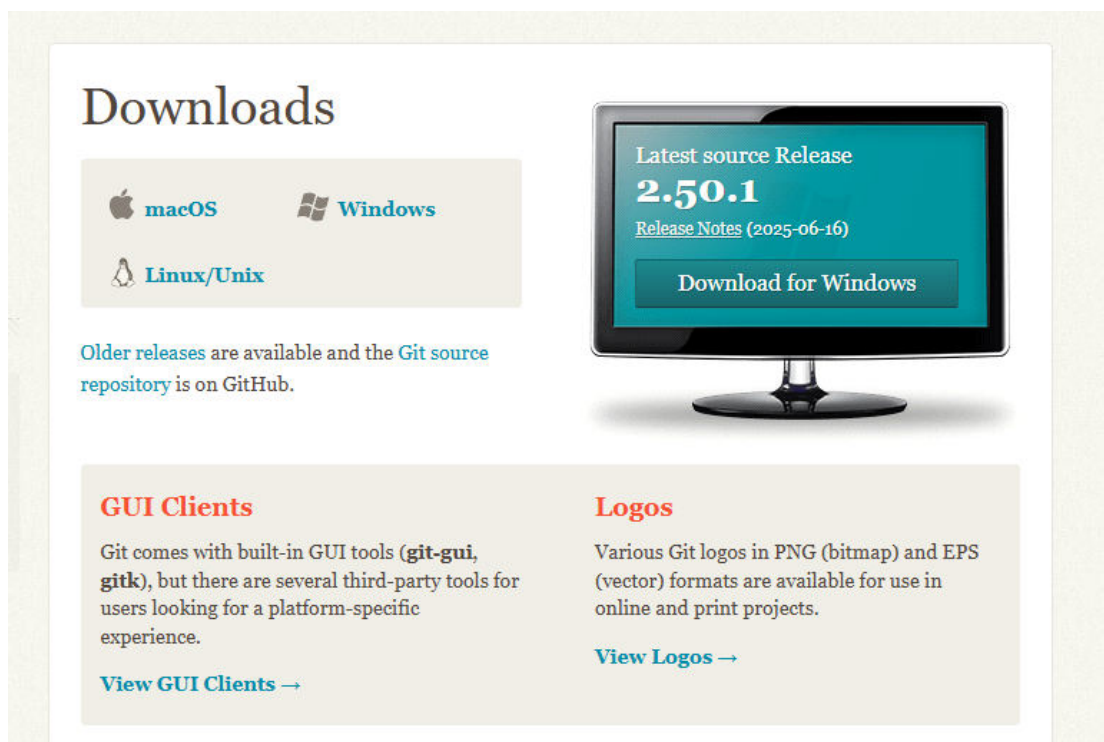
Features of Git.

- **Distributed Architecture:** Every user has a complete copy of the entire repository, including its full history. This allows offline work and faster operations.
- **Branching and Merging:** Git makes it easy to create, manage, and merge branches. Developers can work on different features or fixes independently and merge their changes seamlessly.

- **Speed and Performance:** Git is designed to be fast, handling large projects efficiently with quick commits, branching, and merging.
- **Data Integrity:** Git uses SHA-1 hashing to ensure the integrity of the data, so changes are tracked securely and reliably.
- **Staging Area:** Git provides a staging area (or index) where you can prepare changes before committing them, allowing you to control what exactly goes into each commit.
- **Support for Non-Linear Development:** Git's branching model supports complex workflows with multiple branches and parallel development.

Installation of Git.

S1) Goto <https://git-scm.com/downloads>



S2) Click on Download For Windows and select Desired Installer

Download for Windows

[Click here to download](#) the latest (2.50.1) x64 version of **Git for Windows**. This is the most recent maintained build. It was released **16 days ago**, on 2025-07-08.

Other Git for Windows downloads

Standalone Installer

[Git for Windows/x64 Setup](#).

[Git for Windows/ARM64 Setup](#).

Portable ("thumbdrive edition")

[Git for Windows/x64 Portable](#).

[Git for Windows/ARM64 Portable](#).

Using winget tool

Install [winget tool](#) if you don't already have it, then type this command in command prompt or Powershell.

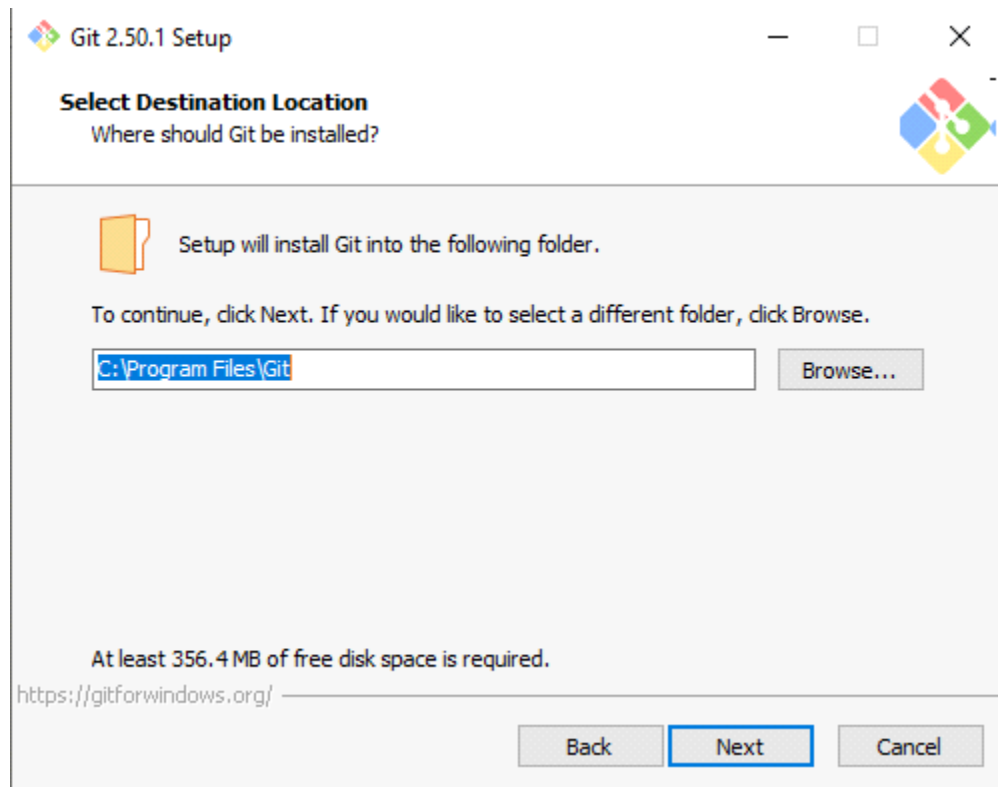
```
winget install --id Git.Git -e --source winget
```

The current source code release is version **2.50.1**. If you want the newer version, you can build it from [the source code](#).

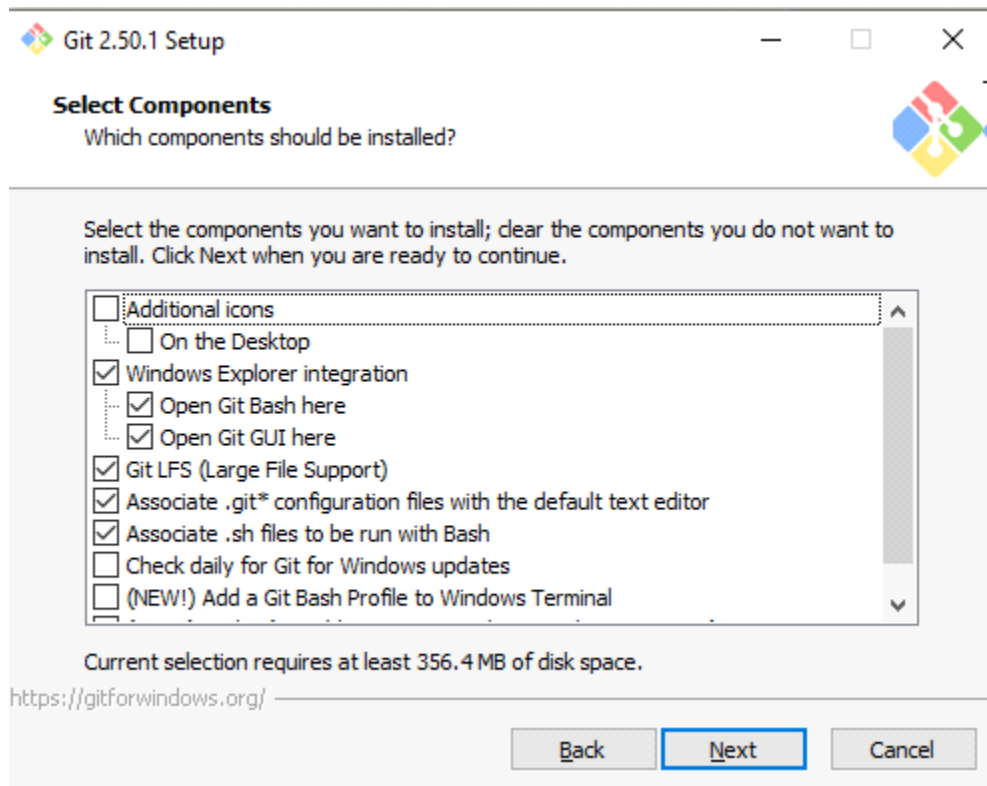
S3) Open Installer and follow steps



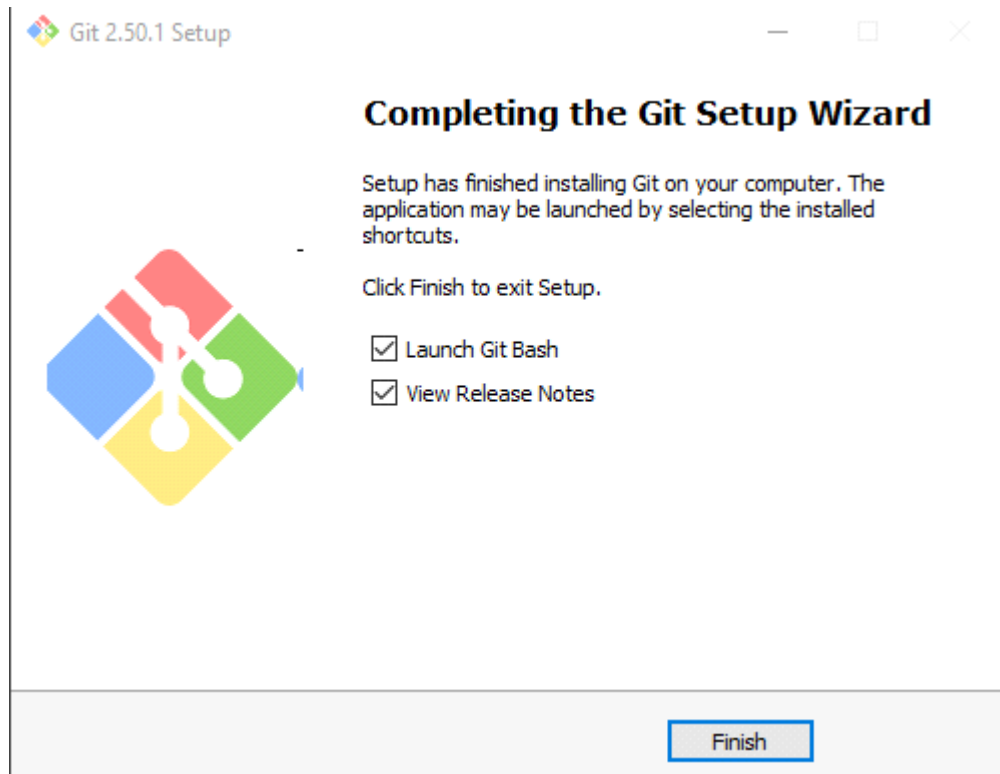
S4) Select Destination Where you want to Install Git



S5) Select Components you want to Install



S6) Launch Git Bash after Installation Complete



Steps for Account Creation in GitHub.

S1) SignUp for GitHub by going to <https://github.com/>

Create your free account

Explore GitHub's core features for individuals and organizations.

[See what's included](#) ▾



[Already have an account?](#) [Sign in](#) →

Sign up to GitHub

Email^{*}

Password^{*}

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username^{*}

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Your Country/Region^{*}

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

S2) Verify Account from Email and SignIn with account created



Sign in to GitHub

Your account was created successfully! Please [✕](#)
sign in to continue.

Username or email address

Password

[Forgot password?](#)

Sign in

or

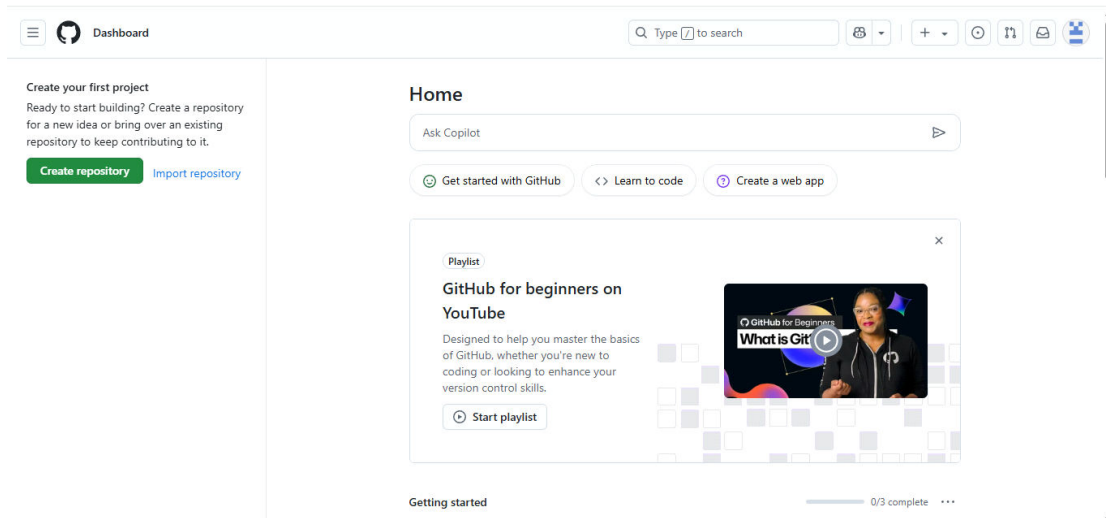


Continue with Google

New to GitHub? [Create an account](#)

[Sign in with a passkey](#)

S3) LogIn to Your Github Account using your Email



Conclusion:

In conclusion, Git is a powerful and flexible version control system that has become essential for modern software development.

Its distributed nature, efficient branching and merging capabilities, and strong data integrity features make it ideal for both individual developers and large teams.

LO Map:

LO1 and LO2

Assignment 3

Aim: To perform various git operations

1) Configure your name and email in git

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git config --global user.email "dhopetanishk4@gmail.com"

lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git config --global user.name "Tanishk Dhope"
```

2) Initialize git repository in your directory

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git init
Reinitialized existing Git repository in C:/Users/lab1002/Tanishk/.git/

lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ |
```

3) **git add** stages all changes (new, modified, and deleted files) in the current directory and its subdirectories for the next commit.

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git add .
warning: in the working copy of 'demo.txt', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'hello.py', LF will be replaced by CRLF the next
time Git touches it
```

4)

git status shows the current state of the working directory and staging area, including which changes are staged, unstaged, or untracked.

git commit -m "message" creates a new commit with the staged changes and attaches the provided commit message.

```

lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   demo.txt
        new file:   hello.py

lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git add .

lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git commit -m "First Commit"
[master (root-commit) 4419f30] First Commit
 2 files changed, 2 insertions(+)
 create mode 100644 demo.txt
 create mode 100644 hello.py

```

5) `git log --all -p` shows the full commit history with diffs for all branches.

```

lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git log -a -p
commit 4419f3023537837ede35dd4a0b5cb5a9ca765605 (HEAD -> master, main)
Author: Tanishk Dhope <dhopetanishk4@gmail.com>
Date: Thu Jul 31 15:00:01 2025 +0530

    First Commit

diff --git a/demo.txt b/demo.txt
new file mode 100644
index 0000000..c64d5e7
--- /dev/null
+++ b/demo.txt
@@ -0,0 +1 @@
+This is a Demo File.
diff --git a/hello.py b/hello.py
new file mode 100644
index 0000000..ad35e5a
--- /dev/null
+++ b/hello.py
@@ -0,0 +1 @@
+print("Hello World")

```

6) `git branch branchname` creates a new branch with the specified name, but does not switch to it.`

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git branch tanishk

lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git branch
main
* master
tanishk
```

7) `git switch branchname` switches to the specified branch in your Git repository.

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git switch tanishk
Switched to branch 'tanishk'

lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git add .

lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git commit -m "Commit on branch Tanishk"
On branch tanishk
nothing to commit, working tree clean
```

8) `git log` gives log of all commits in current repository

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git log --all --graph
* commit ca97fbe7e181f545b8372f297dfb3b183485aba5 (HEAD -> tanishk)
| Author: Tanishk Dhope <dhopetanishk4@gmail.com>
| Date: Thu Jul 31 15:10:30 2025 +0530
|
| NEW
|
| * commit 2576ca2975aa11a950ddaa5ecfb5f1acab83acbb
| | Author: Tanishk Dhope <dhopetanishk4@gmail.com>
| | Date: Thu Jul 31 15:08:43 2025 +0530
| |
| | new file
| |
| * commit 4419f3023537837ede35dd4a0b5cb5a9ca765605 (master, main)
| | Author: Tanishk Dhope <dhopetanishk4@gmail.com>
| | Date: Thu Jul 31 15:00:01 2025 +0530
| |
| | First Commit
```

9) `git rm` removes files from the working directory and stages the removal for the next commit.


```
lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git rm hello.py
rm 'hello.py'
```

10)

``git stash`` temporarily saves uncommitted changes in a stack and reverts the working directory to a clean state.

``git stash list`` shows a list of all stashed changes.

``git stash pop`` applies the most recently stashed changes and removes them from the stash list.

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git stash
Saved working directory and index state WIP on tanishk: ca97fbe NEW

lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git stash list
stash@{0}: WIP on tanishk: ca97fbe NEW

lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git stash pop
On branch tanishk
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    hello.py

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (76ccdc45779bdf5c785c396484d13edebf66a46)
```

11) ``git mv`` moves or renames a file and stages the change for the next commit.

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ mkdir newFolder

lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ ls
demo.txt  new.html  newFolder/  newnew.txt

lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git mv new.html newFolder/new.html
```

12) ``git remote add origin <url>`` adds a remote repository with the name ``origin`` and the specified URL.

``git remote -v`` lists all configured remote repositories and their fetch/push URLs.

```
lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git remote add origin https://github.com/DhopeTanishk/Tanishk_Demo.git

lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git remote -v\
> ;
origin https://github.com/DhopeTanishk/Tanishk_Demo.git (fetch)
origin https://github.com/DhopeTanishk/Tanishk_Demo.git (push)
```

- 13) ``git push origin branch`` uploads the local branch to the remote repository named ``origin``.

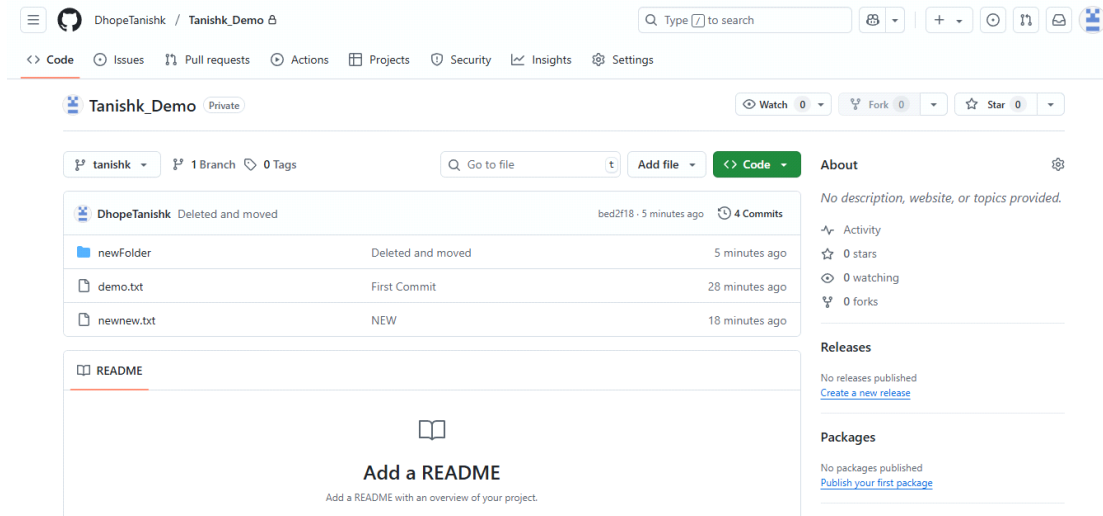
```
lab1002@LAB2-38 MINGW64 ~/Tanishk (tanishk)
$ git push origin tanishk
info: please complete authentication in your browser...
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (13/13), 1.07 KiB | 274.00 KiB/s, done.
Total 13 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/DhopeTanishk/Tanishk_Demo.git
 * [new branch]      tanishk -> tanishk
```

- 14) ``git pull`` fetches changes from the remote repository and merges them into the current local branch.

```
lab1002@LAB2-38 MINGW64 ~/Tanishk
$ git init
Initialized empty Git repository in C:/Users/lab1002/Tanishk/.git/

lab1002@LAB2-38 MINGW64 ~/Tanishk (master)
$ git pull https://github.com/DhopeTanishk/Tanishk_Demo
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 13 (delta 2), reused 13 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (13/13), 1.05 KiB | 2.00 KiB/s, done.
From https://github.com/DhopeTanishk/Tanishk_Demo
 * branch            HEAD            -> FETCH_HEAD
```

- 15) Online Repository pushed from local Repository



Conclusion:

Git is a powerful version control system that helps developers manage and track changes in their code. The commands you've explored—such as ``git add``, ``git commit``, ``git push``, ``git pull``, ``git branch``, ``git stash``, and others—form the foundation of daily Git operations. They allow you to stage changes, create snapshots of your work, collaborate with others through remote repositories, and manage different development branches.

Understanding how these commands work together enables smoother workflows, reduces the risk of losing work, and enhances collaboration in team environments. Mastery of these basic Git commands is essential for efficient and organized software development.

LO Map: LO1 AND LO2

Assignment 4

Aim: To understand continuous Integration and Jenkins Installation

Theory:

Jenkins is an open-source automation tool widely used for managing and automating the software development process. It allows developers to set up continuous integration and continuous delivery (CI/CD) pipelines, helping to automate tasks such as building, testing, and deploying code. Jenkins supports numerous plugins, making it highly customizable and capable of integrating with a wide range of tools and technologies in the development ecosystem.

Continuous integration is a software development practice where developers regularly commit their code to a shared repository, often multiple times a day. Each commit triggers automated builds and tests, ensuring that new changes do not break the existing functionality. This practice helps identify bugs early, improves collaboration among team members, and enables faster and more reliable software releases.

Some key features of Jenkins in short:

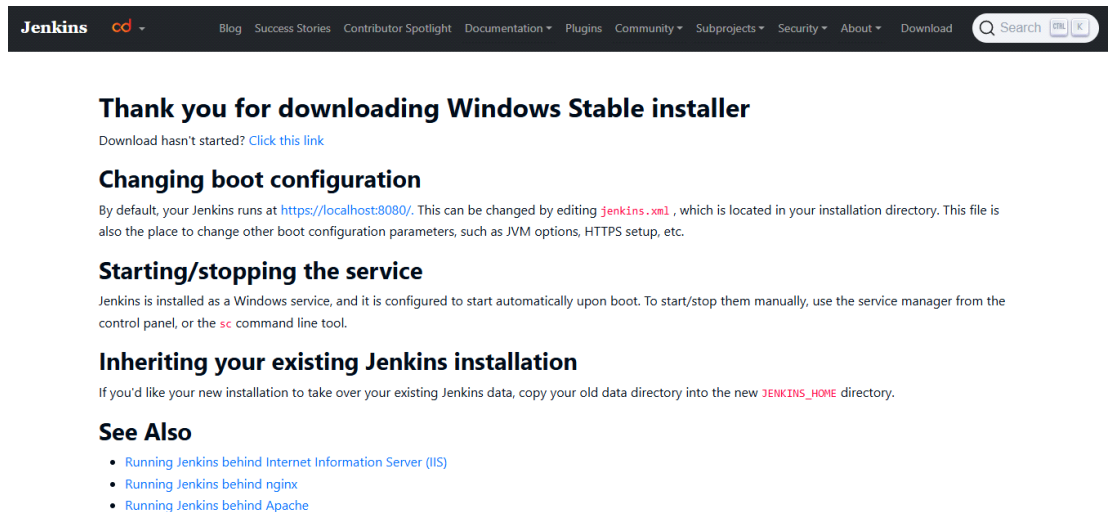
- * Open-source and free to use
- * Supports automation of build, test, and deployment processes
- * Highly extensible with a vast library of plugins
- * Easy to configure through a web-based interface
- * Supports distributed builds across multiple machines
- * Integrates with many version control systems and tools
- * Provides real-time feedback and reporting on build status

Jenkins Installation

- 1) Goto jenkins website <https://www.jenkins.io/>



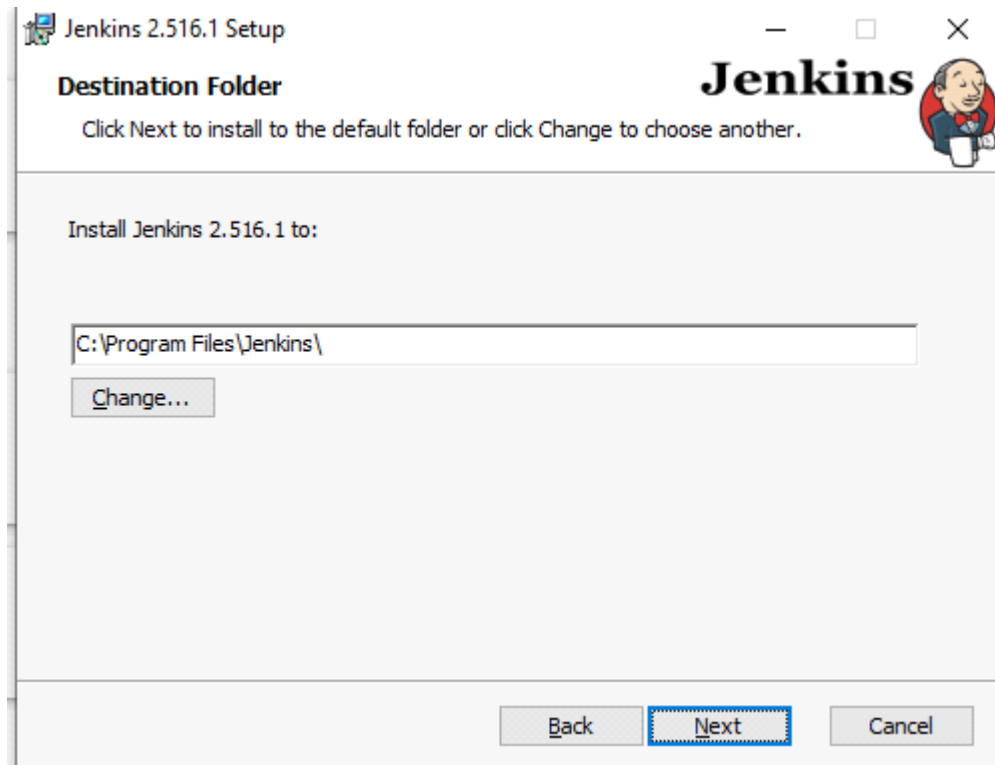
2) Choose Stable LTS version and select windows installer.



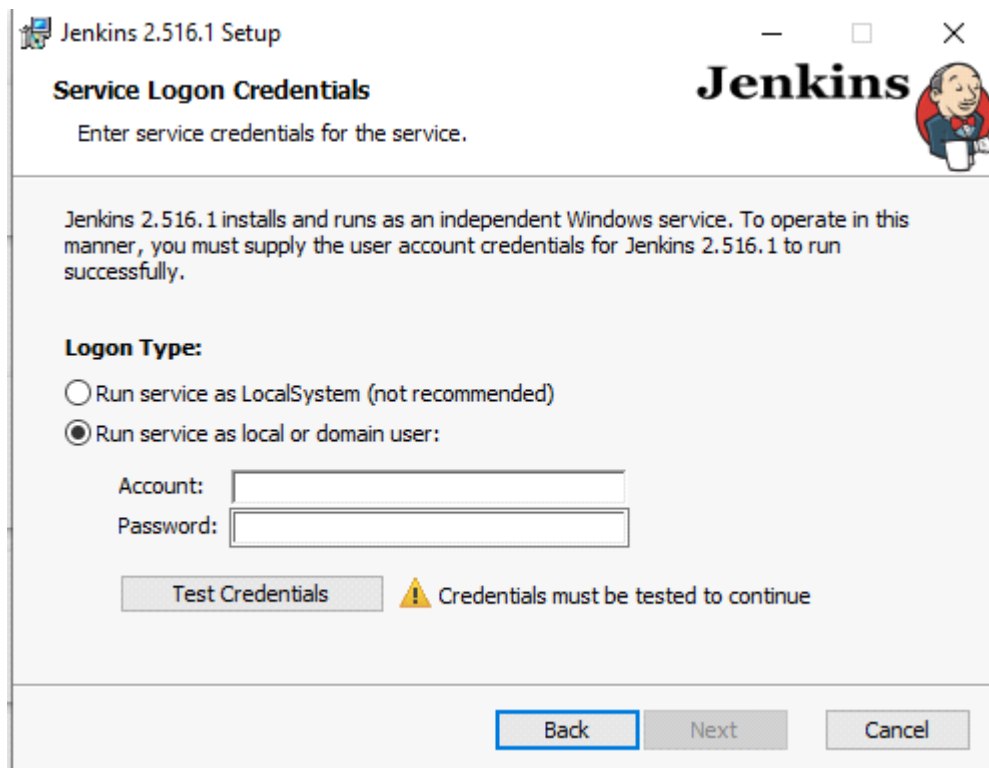
3) Open Jenkins Installer and follow installation steps



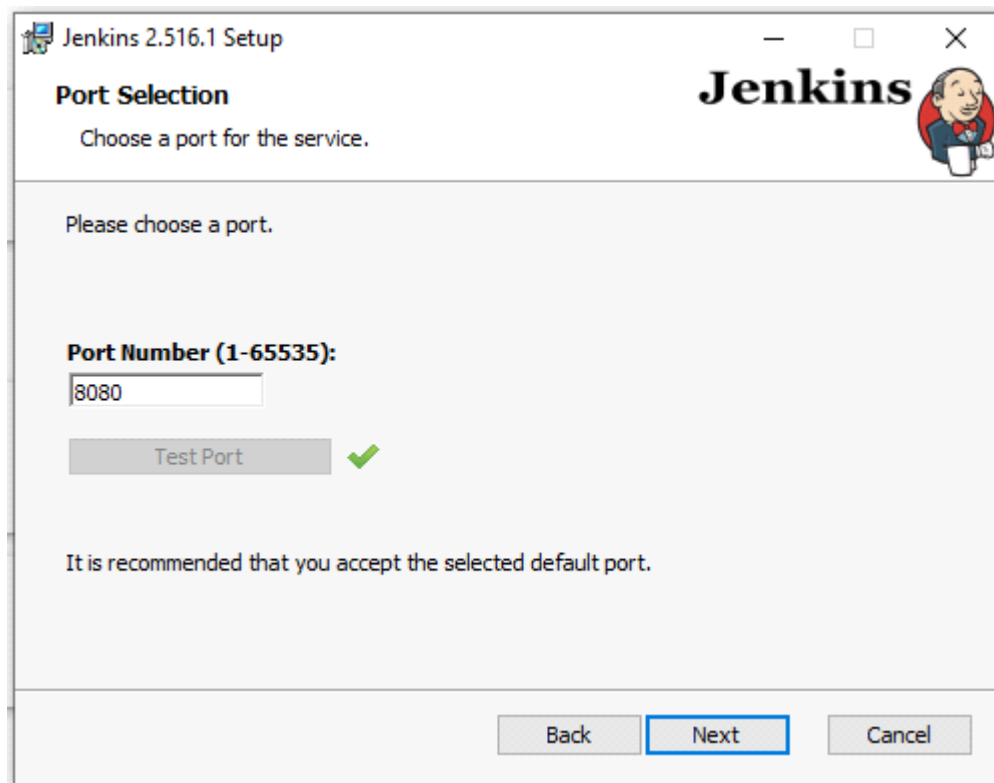
4) Select destination to install jenkins



5) Click on Run Service as LocalSystem



6) Select Desired Port Number



The image shows a Windows-style dialog box titled "Jenkins 2.516.1 Setup". The title bar includes standard minimize, maximize, and close buttons. The dialog has a header section with the word "Jenkins" in a large, bold, serif font, accompanied by a cartoon mascot of a man with a mustache holding a cup. Below the header, the text "Port Selection" is displayed, followed by the instruction "Choose a port for the service." The main area of the dialog contains the text "Please choose a port." and a label "Port Number (1-65535):" above a text input field containing the number "8080". Below the input field is a button labeled "Test Port" followed by a green checkmark icon. At the bottom of the dialog, there is a row of three buttons: "Back", "Next" (which is highlighted with a blue border), and "Cancel". A note at the bottom of the main area states, "It is recommended that you accept the selected default port."

Jenkins 2.516.1 Setup

Jenkins

Port Selection

Choose a port for the service.

Please choose a port.

Port Number (1-65535):

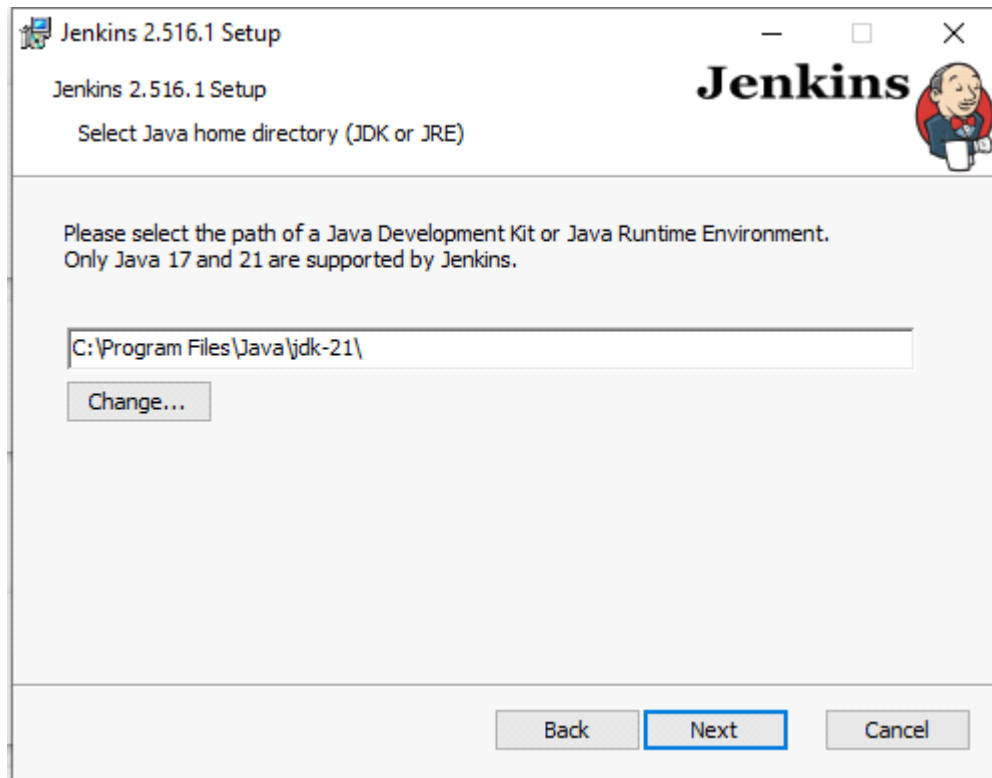
8080

Test Port ✓

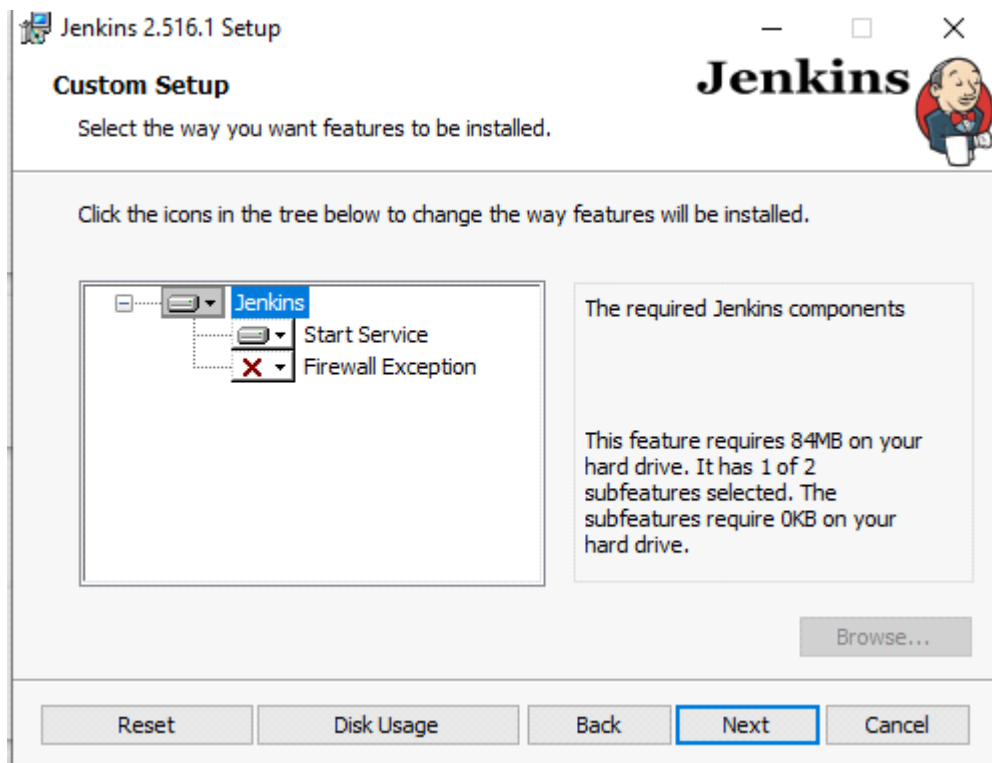
It is recommended that you accept the selected default port.

Back Next Cancel

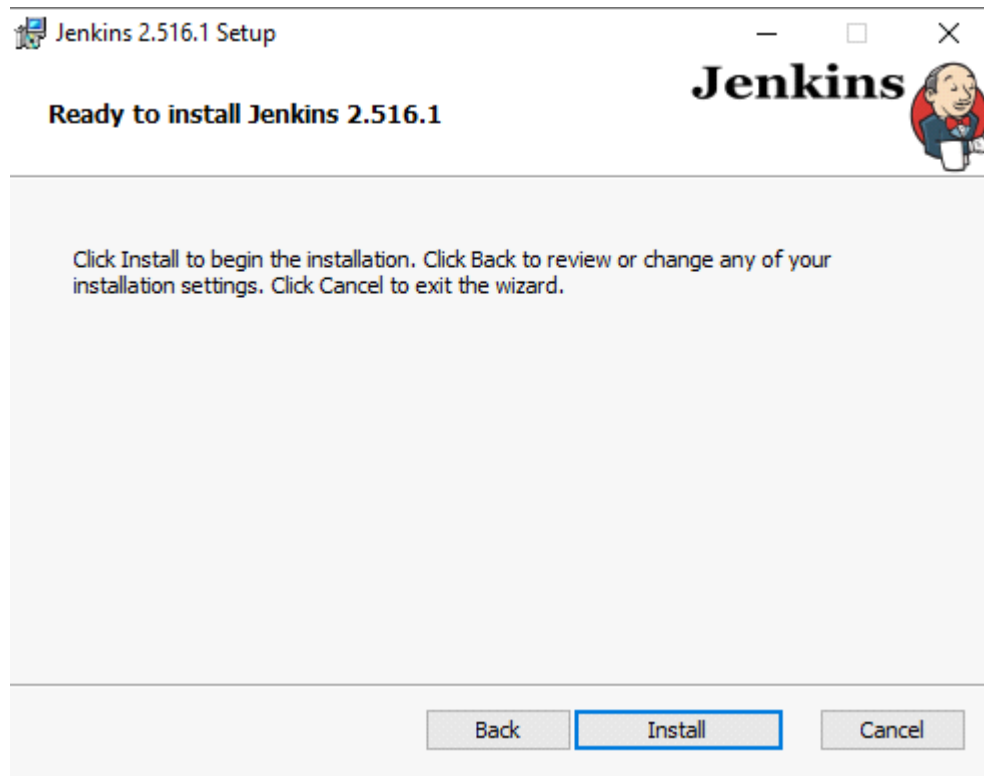
7) select jdk installation path



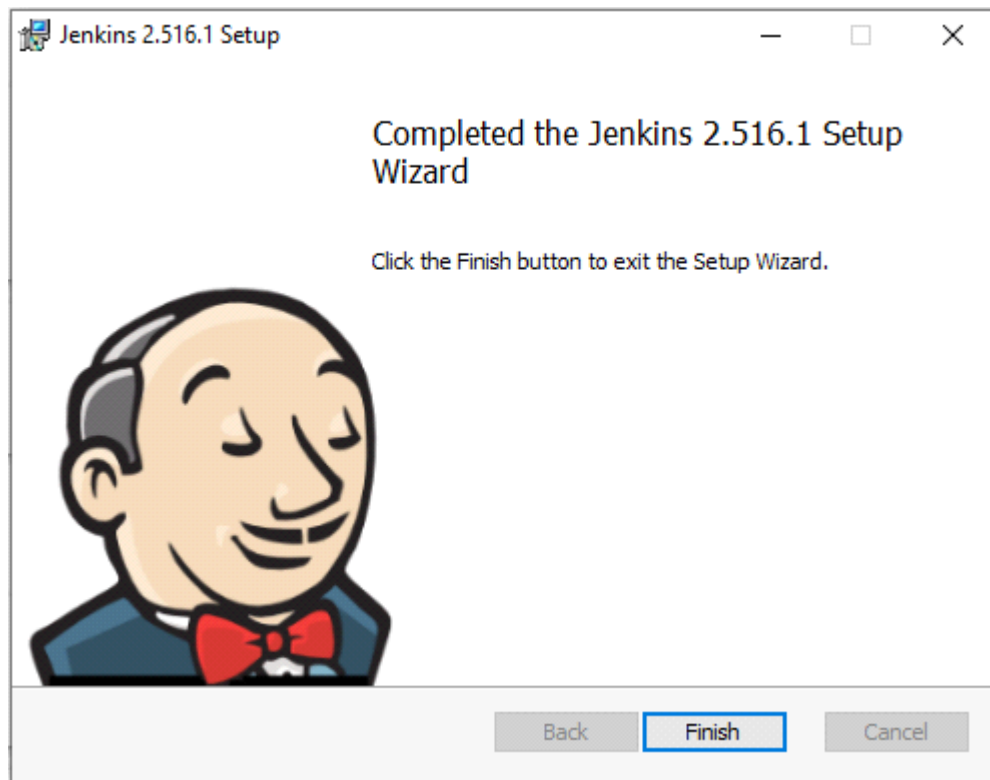
8)



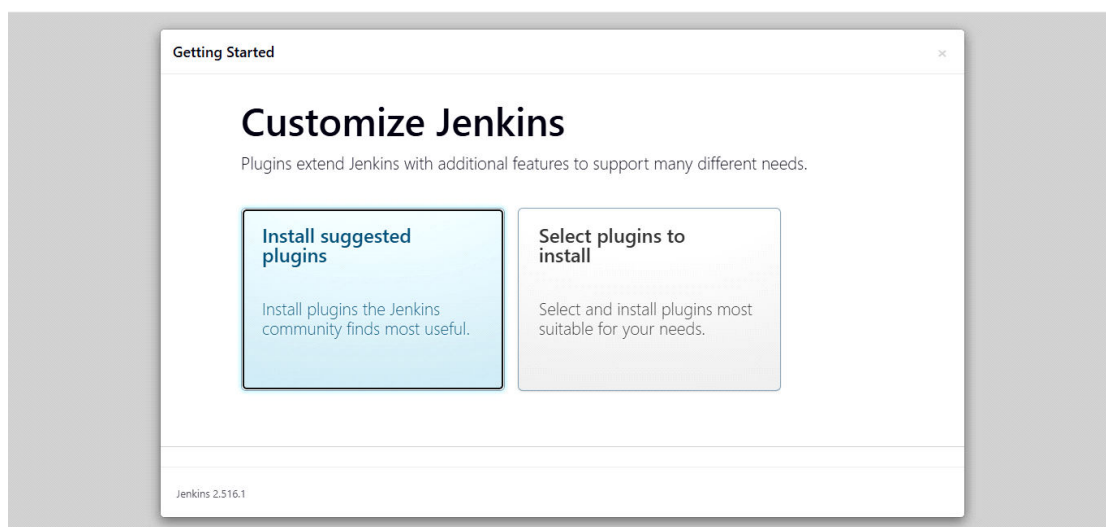
9) Click on install jenkins



10) Click on Finish Wizard



11)



12)

Getting Started

Create First Admin User

Username

Password

Confirm password

Full name

Jenkins 2.516.1

Skip and continue as admin

Save and Continue

13)

Getting Started

Jenkins is ready!

You have skipped the **setup of an admin user**.

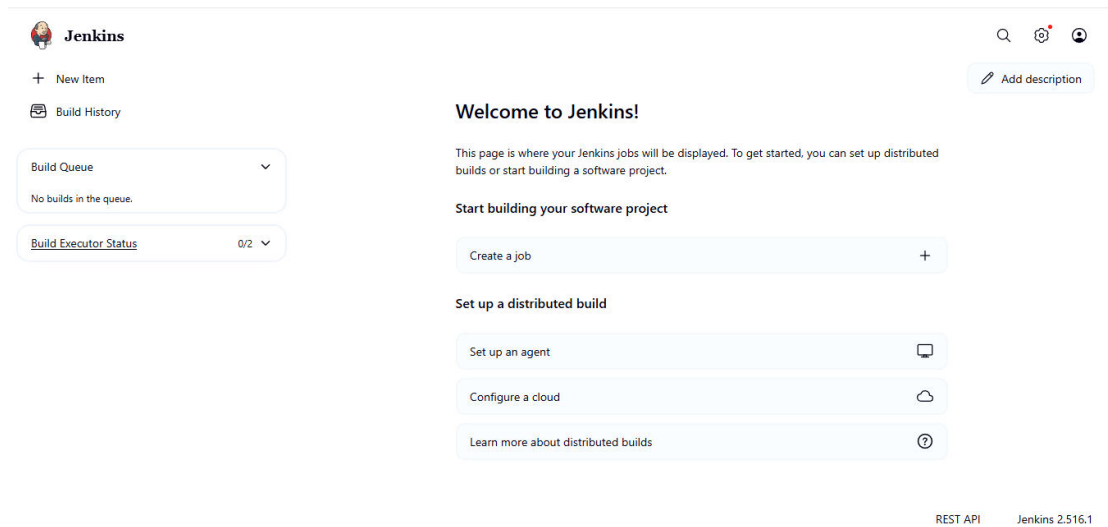
To log in, use the username: "admin" and the administrator password you used to access the setup wizard.

Your Jenkins setup is complete.

Start using Jenkins

Jenkins 2.516.1

14)



Conclusion:

In conclusion, both Jenkins and continuous integration play a crucial role in modern software development by promoting automation, collaboration, and early detection of issues. By integrating tools like Jenkins into the development workflow, teams can streamline their processes, maintain high code quality, and accelerate the delivery of reliable software products.

LO Map: LO4

Assignment 5

Aim: To build a java program with the help of Jenkins

Theory:

Jenkins is an open-source automation server widely used for continuous integration (CI) and continuous delivery/deployment (CD). It allows developers to automate the building, testing, and deployment of software projects.

It's an automation server that takes over the repetitive tasks of building, testing, and preparing your software for release, allowing developers to focus on writing code.

When integrating a Java program with Jenkins, the goal is to automate the compilation, execution, and possibly testing of the Java code each time changes are pushed to the repository or a build is triggered. Jenkins supports this by allowing users to define build steps using shell scripts, batch files, or pipeline scripts.

Key Concepts:

Java Program:

A Java program typically involves writing source code (.java), compiling it using javac to generate bytecode (.class), and running it using the java command.

Jenkins Workspace:

When a job runs, Jenkins checks out the project into a specific workspace directory (e.g., C:\ProgramData\Jenkins\.jenkins\workspace\your-job-name).

Build Steps:

Jenkins jobs can include build steps like:

- Compile Java code: **javac YourProgram.java**
- Run the program: **java YourProgram**

JDK Configuration:

Jenkins needs access to a valid Java Development Kit (JDK). This can be set globally or per-job within Jenkins configuration.

Build Triggers:

Jobs can be triggered manually, on a schedule, or automatically based on events like code commits.

HelloWorld - Notepad

File Edit Format View Help

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```



Jenkins / javanew



Status



Changes



Workspace



Build Now



Configure



Delete Project



Rename



javanew

this is my project

Permalinks

- [Last build \(#2\), 3 min 2 sec ago](#)
- [Last stable build \(#2\), 3 min 2 sec ago](#)
- [Last successful build \(#2\), 3 min 2 sec ago](#)
- [Last failed build \(#1\), 3 min 57 sec ago](#)
- [Last unsuccessful build \(#1\), 3 min 57 sec ago](#)
- [Last completed build \(#2\), 3 min 2 sec ago](#)

Builds



Filter



Jenkins / javanew / #2 / Console Output



Status



Changes



Console Output



Edit Build Information



Delete build '#2'



Timings



Previous Build



Console Output

Download

```
Started by user admin  
Running as SYSTEM  
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\javanew  
[javanew] $ cmd /c call C:\Windows\TEMP\jenkins14024374249335739388.bat  
  
C:\ProgramData\Jenkins\.jenkins\workspace\javanew>javac HelloWorld.java  
  
C:\ProgramData\Jenkins\.jenkins\workspace\javanew>java HelloWorld  
Hello, World!  
  
C:\ProgramData\Jenkins\.jenkins\workspace\javanew>exit 0  
Finished: SUCCESS
```



Jenkins

/ All

/ New Item

New Item

Enter an item name

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Conclusion:

Running a Java program in Jenkins demonstrates the power of automation in software development. By configuring Jenkins to compile and execute Java code, teams can streamline their build and test processes, leading to faster development cycles and fewer human errors.

LO Mapping: LO6

ASSIGNMENT 6

Aim: To build pipeline using Jenkins.

Theory:

What is a Pipeline in Jenkins?

A Jenkins Pipeline is a set of automated processes that help developers and DevOps teams build, test, and deploy code in a continuous integration/continuous delivery (CI/CD) environment.

In simple terms, it's a workflow that defines how software moves from code to production in an automated and repeatable manner.

Key Benefits of Jenkins Pipeline:

- Automates CI/CD processes
- Provides better visibility into the build process
- Offers version control of build logic (Pipeline as Code)
- Supports complex workflows (parallel execution, conditional logic, etc.)

Types of Jenkins Pipelines:

There are two main types of syntax used to define Jenkins pipelines:

1. Declarative Pipeline

Simpler and more structured syntax

Designed for users who want an easy-to-read format

Introduced later to make pipelines more readable and maintainable

Key Features:

- Uses pipeline {} block
- Organized into stages and steps
- Easier error handling
- Better validation in Jenkins UI

2. Scripted Pipeline

- More flexible and powerful
- Written in Groovy-based syntax
- Gives full control over the build process
- Suitable for advanced users Key

Features:

- Uses node {} block
- Everything is written as code
- Allows complex scripting and logic
- Less user-friendly for beginners

1. Declarative Pipeline

CREATION:

New Item

New Item

Enter an item name

decPipeline1

» A job already exists with the name 'decPipeline1'

Select an item type

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

 **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 **Folder**

OK

CODE:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building..'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying...'
            }
        }
    }
}
```

OUTPUT:

```
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\decPipeline1
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building..
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

2. SCRIPTED CREATION:

New Item

New Item

Enter an item name

Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter.

OK

CODE:

```
node {  
  stage('Build') {  
    //  
  }  
  stage('Test') {  
    //  
  }  
  stage('Deploy') {  
    //  
  }  
}
```

OUTPUT:

```
[Pipeline] node  
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\pipeline2  
[Pipeline] {  
[Pipeline] stage  
[Pipeline] { (Build)  
[Pipeline] echo  
Building...  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] stage  
[Pipeline] { (Test)  
[Pipeline] echo  
Testing...  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] stage  
[Pipeline] { (Deploy)  
[Pipeline] echo  
Deploying...  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS
```

Conclusion:

In this assignment, we learned how to create Jenkins Pipelines to automate the build, test, and deployment process. We explored both Declarative and Scripted pipelines, understanding their structure, benefits, and use cases. By implementing different pipeline examples, we demonstrated how Jenkins enables efficient, repeatable, and reliable CI/CD workflows.

LO MAP: LO1,LO3

ASSIGNMENT 7

Aim: To understand Docker architecture and container life cycle, install docker ,deploy container in Docker.

THEORY: Docker is an open-source platform that automates the deployment, scaling, and management of applications inside lightweight, portable containers. Containers allow developers to package an application with all its dependencies and run it reliably in different computing environments.

Docker's architecture consists of the following key components:

Docker Client: The command-line interface where users issue Docker commands.

Docker Daemon: A background service that handles building, running, and monitoring containers.

Docker Images: Immutable templates used to create containers, containing all application code and dependencies.

Docker Containers: Running instances of Docker images, isolated from each other and the host system. Docker Hub: A public cloud repository where Docker images are stored and shared.

The Docker client communicates with the Docker daemon to execute commands. Images are pulled from Docker Hub (or other registries) to create containers that run applications in isolation.

Container Lifecycle

A container undergoes several stages during its lifecycle:

Create: A container is instantiated from a Docker image but not yet running. Start: The container starts executing its processes.

Pause: Temporarily halts container processes.

Unpause: Resumes a paused container.

Stop: Gracefully stops a running container.

Kill: Forces immediate termination of the

container. Remove: Deletes the container from the system.

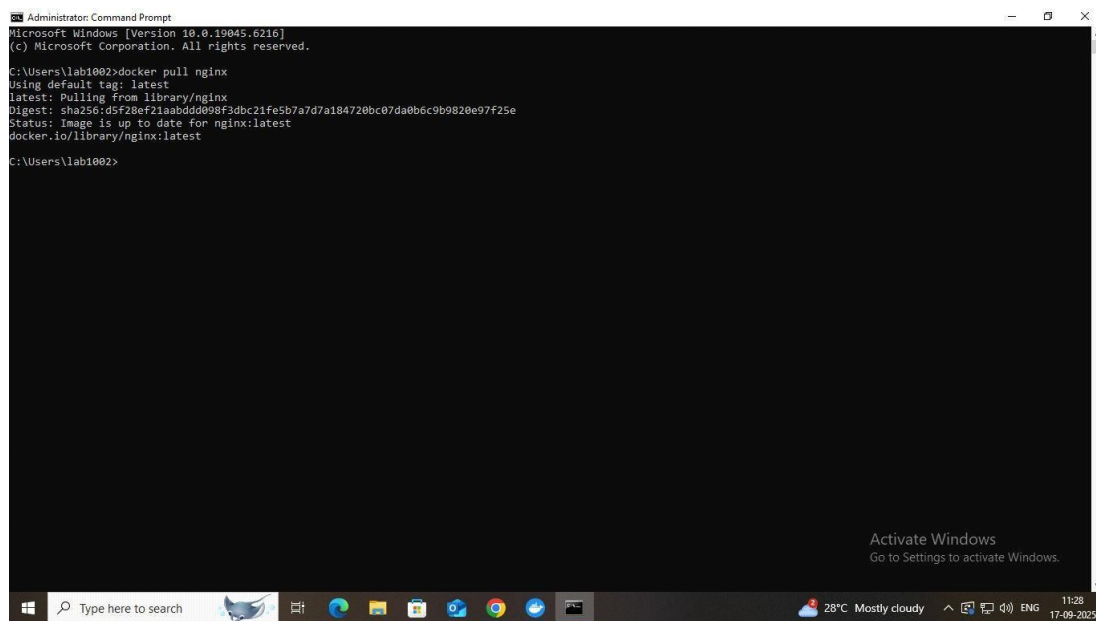
Understanding this lifecycle helps in effectively managing containers.

Deploying a Container in Docker

To deploy a container, following are the steps:

Step 1: Pull the Docker Image

Download the official NGINX image from Docker Hub by running: `docker pull nginx`



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.6216]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lab1002>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:d5f28ef21aabdd098f3dbc21fe5b7a7d7a184720bc07da0b6c9b9820e97f25e
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest

C:\Users\lab1002>
```

The screenshot shows a Windows Command Prompt window titled "Administrator: Command Prompt". The window displays the output of the command `docker pull nginx`. The output indicates that the latest version of the nginx image is being pulled from the Docker Hub library. The digest is `sha256:d5f28ef21aabdd098f3dbc21fe5b7a7d7a184720bc07da0b6c9b9820e97f25e`, and the status is "Image is up to date for nginx:latest". The command prompt shows the user is in the directory `C:\Users\lab1002`. The Windows taskbar at the bottom shows the search bar, task view button, and several application icons. The system tray on the right shows the temperature as 28°C, weather as "Mostly cloudy", and the time as 11:28 on 17-09-2025.

S2: Run the Container

Create and start a container named mynginx with port mapping:

```
docker run -d -p 8081:80 --name mynginx nginx
```

-d runs the container in detached mode.

-p 8081:80 maps port 8081 on the host to port 80 in the container.

--name mynginx assigns a name to the container.

```
Administrator: Command Prompt
Digest: sha256:d5f28ef21aabd098f3dbc21fe5b7a7d7a184720bc07da0b6c9b9820e97f25e
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest

C:\Users\lab1002>docker run -d -p 8082:80 --name T12dockercontainer nginx
500900397af3254d6d57521556d058abf98bb3ca2ae63cbca099363223c5cdfd

C:\Users\lab1002>
```

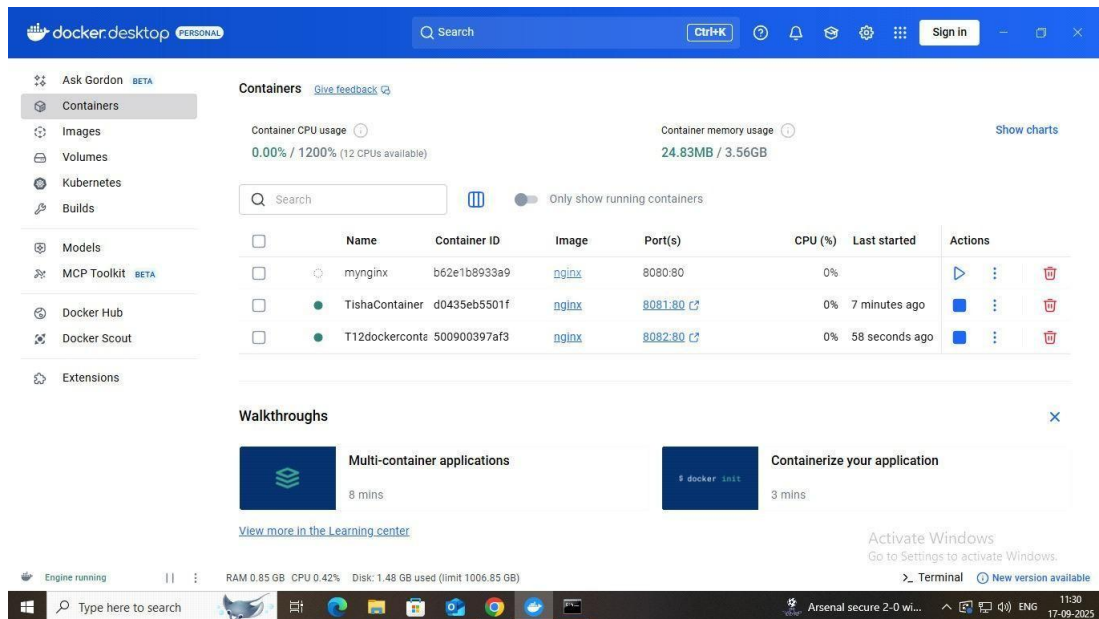
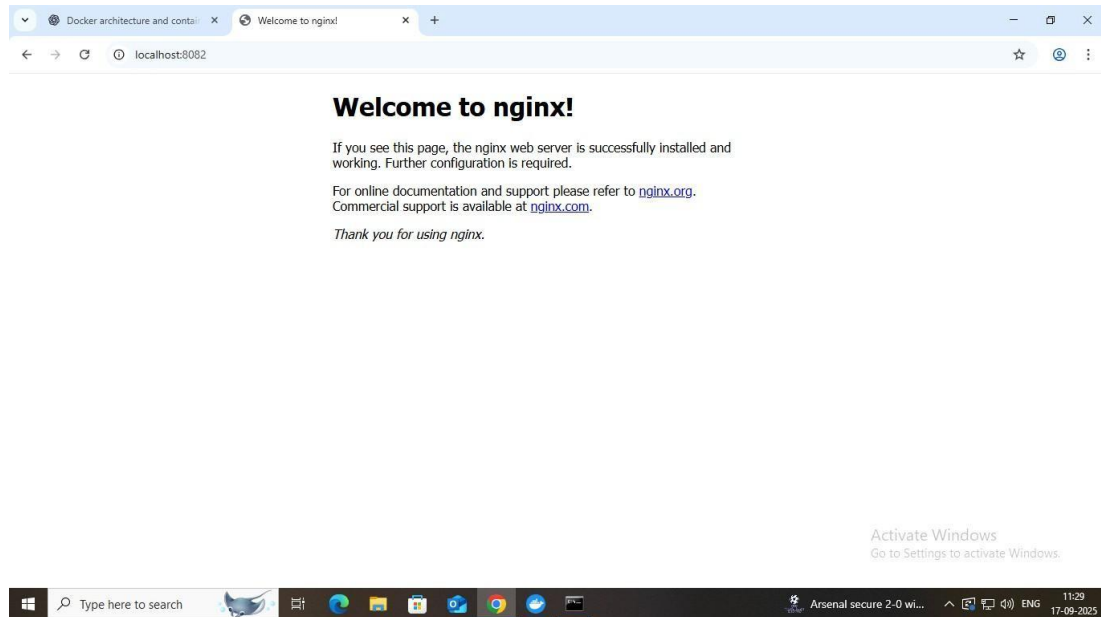
```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.6216]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lab1002>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:d5f28ef21aabd098f3dbc21fe5b7a7d7a184720bc07da0b6c9b9820e97f25e
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest

C:\Users\lab1002>docker run -d -p 8082:80 --name T12dockercontainer nginx
500900397af3254d6d57521556d058abf98bb3ca2ae63cbca099363223c5cdfd

C:\Users\lab1002>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
500900397af3   nginx     "/docker-entrypoint..." 24 seconds ago Up 20 seconds 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp T12dockercontainer
d0435eb5501f   nginx     "/docker-entrypoint..." 6 minutes ago  Up 6 minutes  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp TishaContainer

C:\Users\lab1002>
```



S3: Verify the Running Container Check

active containers with:

`docker ps`

S4: Access the Application Open a

web browser and navigate to:

`http://localhost:8081`

You will see the NGINX welcome page, indicating the container is running successfully.

Conclusion: This assignment demonstrated the basic understanding of Docker's architecture and container lifecycle, as well as practical experience installing Docker and deploying a container. Docker containers provide an efficient and consistent environment to run applications, improving portability and scalability.

LO MAP: LO1, LO5 mapped

ASSIGNMENT 8

Aim: To build an image for sample web application using Dockerfile

Theory:

Steps:

S1: Creating a Dockerfile

A Dockerfile is a text file that contains instructions for building a Docker image. It specifies the base image, sets up the environment, and defines how the application should be installed and configured within the container.

S2: Base Image: The first line of your Dockerfile should specify a base image. The base image provides the foundational operating system and environment for your application. You can use official images from Docker Hub or create custom base images.

Eg: FROM ubuntu:20.04

S3: Setting the Working Directory: You should set a working directory within the container where your application's code will reside. This makes it easier to manage and run your application.

Example: WORKDIR /app

S4: Copying Files: Use the COPY or ADD instructions to copy your application code and any necessary files into the container.

Example:

COPY . /app

S5: Installing Dependencies: If your application requires dependencies like libraries or packages, you can use the container's package manager or run custom commands to install them.

Example (for a Node.js application): RUN npm install

S6: Exposing Ports: If your web application listens on a specific port, use the EXPOSE instruction to specify the port that should be exposed when the container is running.

Example:

EXPOSE 80

S7: Command to Start the Application: Use the CMD or ENTRYPOINT instruction to define the command that should be executed when the container is started. This should be the command that launches your web application.

Example:

CMD ["node", "app.js"]

S8: Docker Build:

To build the Docker image, use the docker build command. Provide the path to the directory containing your Dockerfile and application code.

Example:

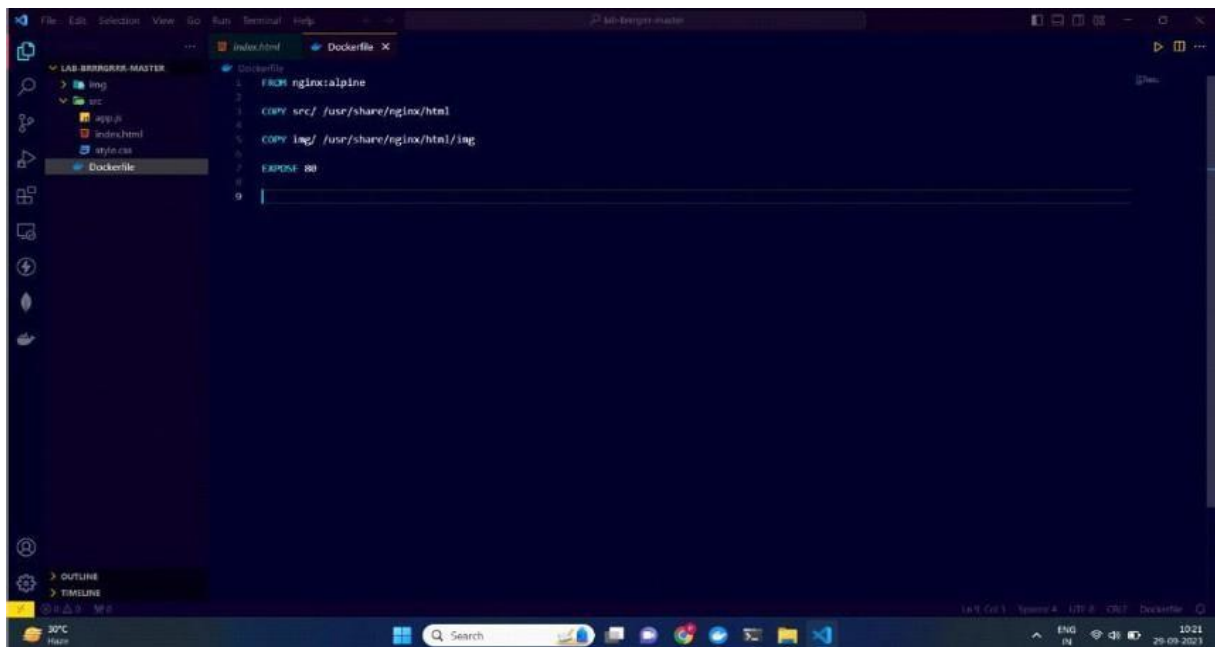
docker build -t my-web-app

S9: Docker run:

To run a container from the image you've built, use the docker run command. Map the container's exposed port to a port on your host machine if needed.

Example:

docker run -p 8080:80 my-web-app




```
C:\Windows\System32\cmd.exe x + v
Microsoft Windows [Version 10.0.22621.2283]
(c) Microsoft Corporation. All rights reserved.

E:\lab-brrrrrr-master>code .

E:\lab-brrrrrr-master>docker build -t sample-web-app .
[+] Building 9.3s (9/9) FINISHED                                docker:default
=> [internal] load dockerignore                                0.1s
=> => transferring context: 2B                                  0.0s
=> [internal] load build definition from Dockerfile            0.1s
=> => transferring dockerfile: 185B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 4.3s
=> [auth] library/nginx:pull token for registry-1.docker.io    0.8s
=> [1/3] FROM docker.io/library/nginx:alpine@sha256:4c93a3bd8bf95412 4.5s
=> => resolve docker.io/library/nginx:alpine@sha256:4c93a3bd8bf95412 0.1s
=> sha256:96526aa774ef8126ad0fe9e9a95764c5fc37f40 3.40MB / 3.40MB 1.1s
=> sha256:f2004135e416117cc29b9fd1a5c217b19bd2555 1.90MB / 1.90MB 1.1s
=> sha256:4c93a3bd8bf95412889dd84213570102176b605 1.65kB / 1.65kB 0.0s
=> sha256:34b58b4f5cd133d97298c8aae148283dc325ff 1.99kB / 1.99kB 0.0s
=> sha256:d571254277f6a8ba9d0c4a88f29b94476dcd4 16.69kB / 16.69kB 0.0s
=> sha256:fbf1cf5026c467c51d6532a304acb35164d5aaee73d 626B / 626B 0.5s
=> sha256:38966af6931dfff98fc0ff3f63f490938a895c2739b20 0.8s
=> sha256:c3ee70732c61e54665d4cd10d75c2962958b72d6dbef 0.8s
=> sha256:76cbc9ea6abf208d8889d7fe3c6ad19d6f0ce9e 1.40kB / 1.40kB 1.4s
=> sha256:7e2fd992447a7948a6090f3c4eb2dd92ad37ae1 1.21kB / 1.21kB 1.4s
=> extracting sha256:96526aa774ef8126ad0fe9e9a95764c5fc37f409ab9e 0.2s
=> sha256:37f8bcf34db7931f3e1386852d3dde3d244cb 12.64MB / 12.64MB 3.3s
=> extracting sha256:f2004135e416117cc29b9fd1a5c217b19bd25556f8f5 0.6s
=> extracting sha256:fbf1cf5026c467c51d6532a304acb35164d5aaee73d5 0.0s
=> extracting sha256:38966af6931dfff98fc0ff3f63f490938a895c2739b20 0.8s
=> extracting sha256:c3ee70732c61e54665d4cd10d75c2962958b72d6dbef 0.8s
=> extracting sha256:7e2fd992447a7948a6090f3c4eb2dd92ad37ae1144d6 0.8s
=> extracting sha256:76cbc9ea6abf208d8889d7fe3c6ad19d6f0ce9e9b0519 0.8s
=> extracting sha256:37f8bcf34db7931f3e1386852d3dde3d244cb54f28aa 0.8s
=> [internal] load build context                                0.1s
=> => transferring context: 99.20kB                             0.1s
=> [2/3] COPY src/ /usr/share/nginx/html                      0.1s
=> [3/3] COPY img/ /usr/share/nginx/html/img                  0.1s
=> exporting to image                                          0.1s
=> => exporting layers                                           0.8s
```

```
C:\Windows\System32\cmd.exe x + v
=> sha256:d571254277f6a8ba9d0c4a88f29b94476dcd4 16.69kB / 16.69kB 0.8s
=> sha256:fbf1cf5026c467c51d6532a304acb35164d5aaee73d 626B / 626B 0.5s
=> sha256:38966af6931dfff98fc0ff3f63f490938a895c2739b20 0.8s
=> sha256:c3ee70732c61e54665d4cd10d75c2962958b72d6dbef 0.8s
=> sha256:76cbc9ea6abf208d8889d7fe3c6ad19d6f0ce9e 1.40kB / 1.40kB 1.4s
=> sha256:7e2fd992447a7948a6090f3c4eb2dd92ad37ae1 1.21kB / 1.21kB 1.4s
=> extracting sha256:96526aa774ef8126ad0fe9e9a95764c5fc37f409ab9e 0.2s
=> sha256:37f8bcf34db7931f3e1386852d3dde3d244cb 12.64MB / 12.64MB 3.3s
=> extracting sha256:f2004135e416117cc29b9fd1a5c217b19bd25556f8f5 0.6s
=> extracting sha256:fbf1cf5026c467c51d6532a304acb35164d5aaee73d5 0.0s
=> extracting sha256:38966af6931dfff98fc0ff3f63f490938a895c2739b20 0.8s
=> extracting sha256:c3ee70732c61e54665d4cd10d75c2962958b72d6dbef 0.8s
=> extracting sha256:7e2fd992447a7948a6090f3c4eb2dd92ad37ae1144d6 0.8s
=> extracting sha256:76cbc9ea6abf208d8889d7fe3c6ad19d6f0ce9e9b0519 0.8s
=> extracting sha256:37f8bcf34db7931f3e1386852d3dde3d244cb54f28aa 0.8s
=> [internal] load build context                                0.1s
=> => transferring context: 99.20kB                             0.1s
=> [2/3] COPY src/ /usr/share/nginx/html                      0.1s
=> [3/3] COPY img/ /usr/share/nginx/html/img                  0.1s
=> exporting to image                                          0.1s
=> => exporting layers                                           0.8s
=> writing image sha256:713c7cdaaf784f542f0c4e31aa5b6951be58cdb7 0.0s
=> naming to docker.io/library/sample-web-app                 0.0s

What's Next?
View a summary of image vulnerabilities and recommendations + docker scout quickview

E:\lab-brrrrrr-master>docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
sample-web-app      latest     713c7cdaaf78  7 seconds ago 42.7MB
myimage             latest     438bb56a50a3  13 hours ago 122MB
ubuntu              latest     c6b84b685f35  6 weeks ago  77.8MB
hello-world         latest     9c7a54a9a43c  4 months ago 13.3kB

E:\lab-brrrrrr-master>docker run -d -p 8080:80 sample-web-app
e4bf6da28d0642a0b54c01578e67f98e0221350832e1b60ea54bc901d8467dc3

E:\lab-brrrrrr-master>
```



Conclusion: Here we understood the steps to build an image of a web application and also we successfully created an image of sample web application.

LO MAP : LO1,LO5

ASSIGNMENT 9

Aim: Installaton of Nagios on Ubuntu system.

Theory:

Nagios Core is a powerful open-source monitoring tool used to monitor systems, networks, and infrastructure. It provides real-time alerting and visualization for servers, services, switches, applications, and more.

Key Features:

- Server & network monitoring
- Email/SMS alerts for failures
- Web interface for visualization
- Extensible with plugins and custom checks

Prerequisites

- A Linux machine (Ubuntu/Debian-based system)
- Root or `sudo` privileges
- Apache web server
- Basic networking knowledge

Steps:

1. Update System & Install Dependencies

```
sudo apt update
sudo apt install -y apache2 php libapache2-mod-php build-essential \
libgd-dev unzip curl wget libssl-dev daemon make gcc
```

2. Create Nagios User and Group

```
sudo useradd nagios
sudo groupadd nagcmd
sudo usermod -aG nagcmd nagios
sudo usermod -aG nagcmd www-data
```

3. Download and Compile Nagios Core

```
cd /tmp
wget https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.5.1.tar.gz
tar -xzf nagios-4.5.1.tar.gz
cd nagios-4.5.1
./configure --with-command-group=nagcmd
make all
sudo make install
```

```
sudo make install-commandmode
sudo make install-init
sudo make install-config
sudo make install-webconf
```

. Set Up Web Interface Authentication

Create a login user for the web interface (e.g. `admin`):

```
sudo htpasswd -c /usr/local/nagios/etc/htpasswd.users admin
```

Note: Choose a strong password and remember it.

5. Install Nagios Plugins

```
cd /tmp
wget https://nagios-plugins.org/download/nagios-plugins-2.3.3.tar.gz
tar -xzf nagios-plugins-2.3.3.tar.gz
cd nagios-plugins-2.3.3
./configure --with-nagios-user=nagios --with-nagios-group=nagios
make
sudo make install
```

6. Enable Required Apache Modules

```
sudo a2enmod rewrite cgi
sudo systemctl restart apache2
```

7. Start Nagios Service

```
bash
sudo systemctl enable nagios
sudo systemctl start nagios
```

Accessing the Nagios Web Interface

Steps:

1. Get your server IP:

```
ip a | grep inet
```

Example output: `192.168.4.104`

2. Open a browser and visit:

<http://192.168.4.104/nagios>

3. Login with:

Username: `admin`

Password: (the one you set earlier)

```
lab1002@lab1002-HP-280-G3-MT:~$ sudo apt update
Hit:1 https://download.docker.com/linux/ubuntu bionic InRelease
Hit:2 http://security.ubuntu.com/ubuntu bionic-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu bionic InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu bionic-updates InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu bionic-backports InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
47 packages can be upgraded. Run 'apt list --upgradable' to see them.
lab1002@lab1002-HP-280-G3-MT:~$ sudo apt install -y autoconf gcc make unzip apache2 php libapache2-mod-php \
> libgd-dev libmcrypt-dev libssl-dev daemon wget bc gawk dc build-essential \
> snmp libnet-snmp-perl gettext
Reading package lists... Done
Building dependency tree
Reading state information... Done
bc is already the newest version (1.07.1-2).
bc set to manually installed.
build-essential is already the newest version (12.4ubuntu1).
```

```
lab1002@lab1002-HP-280-G3-MT:~$ sudo usermod -a -G nagcmd nagios
lab1002@lab1002-HP-280-G3-MT:~$ sudo usermod -a -G nagcmd www-data
lab1002@lab1002-HP-280-G3-MT:~$ ip a | grep inet
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
    inet 192.168.4.104/24 brd 192.168.4.255 scope global dynamic noprefixroute eth0
    inet6 fe80::59a0:c15b:f605:a620/64 scope link noprefixroute
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
lab1002@lab1002-HP-280-G3-MT:~$ ls /etc/apache2/sites-enabled/nagios.conf
/etc/apache2/sites-enabled/nagios.conf
lab1002@lab1002-HP-280-G3-MT:~$ sudo systemctl reload apache2
lab1002@lab1002-HP-280-G3-MT:~$ ls /usr/local/nagios/share
angularjs          includes           media
bootstrap-3.3.7    index.php         robots.txt
config.inc.php     infobox.html     side.php
contexthelp        js               spin
d3                 jsonquery.html   ssi
docs               locale           stylesheets
graph-header.html  main.php         trends-form.html
histogram-form.html map-directive.html trends-graph.html
histogram-graph.html map-form.html     trends-host-yaxis.html
```

```
lab1002@lab1002-HP-280-G3-MT:~$ cat /usr/local/nagios/etc/htpasswd.users
admin:$apr1$2XweygvH$ReiuYPL/sfq1hcYq9xuMn/
lab1002@lab1002-HP-280-G3-MT:~$ sudo a2enmod rewrite cgi
Module rewrite already enabled
Module cgi already enabled
lab1002@lab1002-HP-280-G3-MT:~$ sudo systemctl restart apache2
lab1002@lab1002-HP-280-G3-MT:~$ sudo htpasswd /usr/local/nagios/etc/htpasswd.users admin
New password:
Re-type new password:
Updating password for user admin
lab1002@lab1002-HP-280-G3-MT:~$ sudo systemctl restart apache2
lab1002@lab1002-HP-280-G3-MT:~$
```

← → ↺ 192.168.4.104/nagios/ ☆

Nagios®

General

- Home
- Documentation

Current Status

- Tactical Overview
- Map (Legacy)
- Hosts
- Services
- Host Groups
 - Summary
 - Grid
- Service Groups
 - Summary
 - Grid
- Problems
 - Services (Unhandled)
 - Hosts (Unhandled)
 - Network Outages

Quick Search:

Reports

- Availability
- Trends (Legacy)
- Alerts
 - History
 - Summary
 - Histogram (Legacy)
- Notifications
- Event Log

System

- Comments
- Downtime
- Dispage Info

Nagios® Core™

✓ Daemon running with PID 1437

Nagios® Core™
Version 4.4.6
April 28, 2020
[Check for updates](#)

Get Started

- Start monitoring your infrastructure
- Change the look and feel of Nagios
- Extend Nagios with hundreds of addons
- Get support
- Get training
- Get certified

Quick Links

- Nagios Library (tutorials and docs)
- Nagios Labs (development blog)
- Nagios Exchange (plugins and addons)
- Nagios Support (tech support)
- Nagios.com (company)
- Nagios.org (project)

Latest News

Don't Miss...

Copyright © 2010-2020 Nagios Core Development Team and Community Contributors. Copyright © 1999-2009 Ethan Galstad. See the THANKS file for more information on contributors.

Page Tour

Conclusion: Hence we have installed Nagios on Ubuntu

LO MAP: LO1,LO5

ASSIGNMENT 10

Aim: To study Puppet Tools.

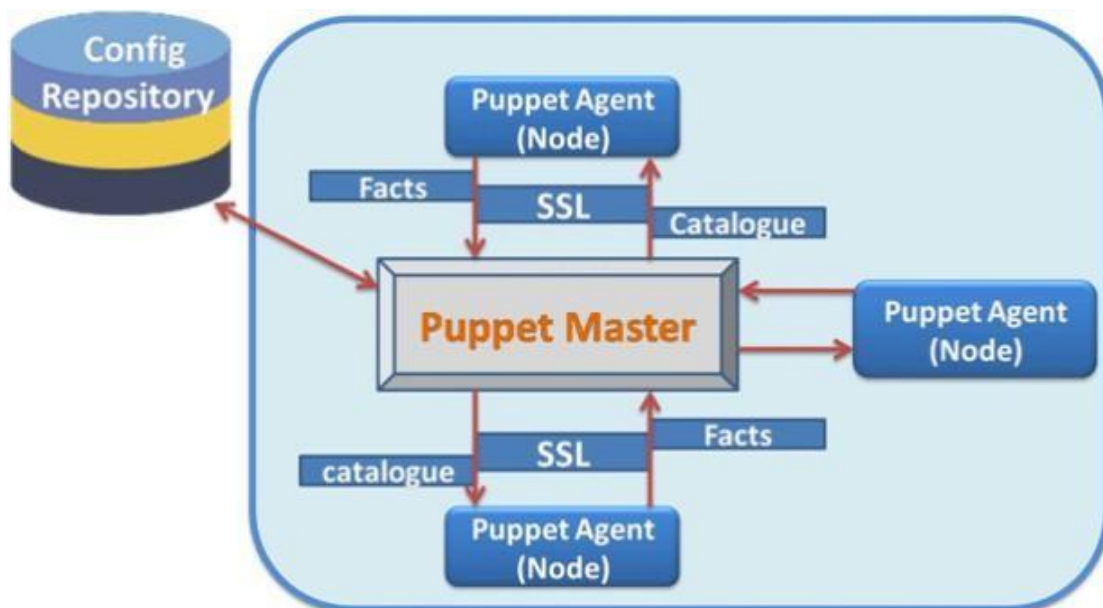
Theory:

1. What is Puppet?

Puppet is an open-source configuration management and automation tool used for managing servers and systems. It allows administrators to:

- Automate repetitive tasks
- Deploy applications
- Manage configurations
- Ensure system consistency across a large number of machines

Puppet uses a **declarative language** to define system configurations, meaning you describe the desired state, and Puppet ensures the system matches that state.



2. Functions of Puppet Tools

- **Configuration Management:** Automatically manages configuration files, users, services, and packages.
- **Automation:** Reduces manual work by automating repetitive system administration tasks.
- **Scalability:** Manages thousands of systems easily with a central control mechanism.
- **Consistency:** Ensures all systems follow the same configuration rules, reducing errors.
- **Compliance & Security:** Ensures systems remain compliant with organizational or regulatory policies.
- **Resource Abstraction:** Provides an abstraction layer so administrators can manage resources without worrying about OS-specific commands.

3. Resources of Puppet Tools

In Puppet, resources are the building blocks used to describe the state of the system.

Classes: A collection of resources that can be reused and included in different manifests.

```
class install_apache { package {  
  'apache2':  
    ensure => installed,  
  }  
  service {  
    'apache2': ensure =>  
      running,      enable  
      => true,  
  }  
}
```

- **Modules:** A directory structure containing manifests, files, templates, and other resources. Helps organize and distribute Puppet code.
Example: An apachemodule may contain manifests to install and configure Apache.

Manifests: Files (with .ppextension) written in Puppet's declarative language that describe the desired system state.

```
file { ['/etc/motd':  
  content => "Welcome to Puppet-managed system!",  
}]
```

- **Nodes:** The client systems managed by Puppet, identified by hostname or certificate.

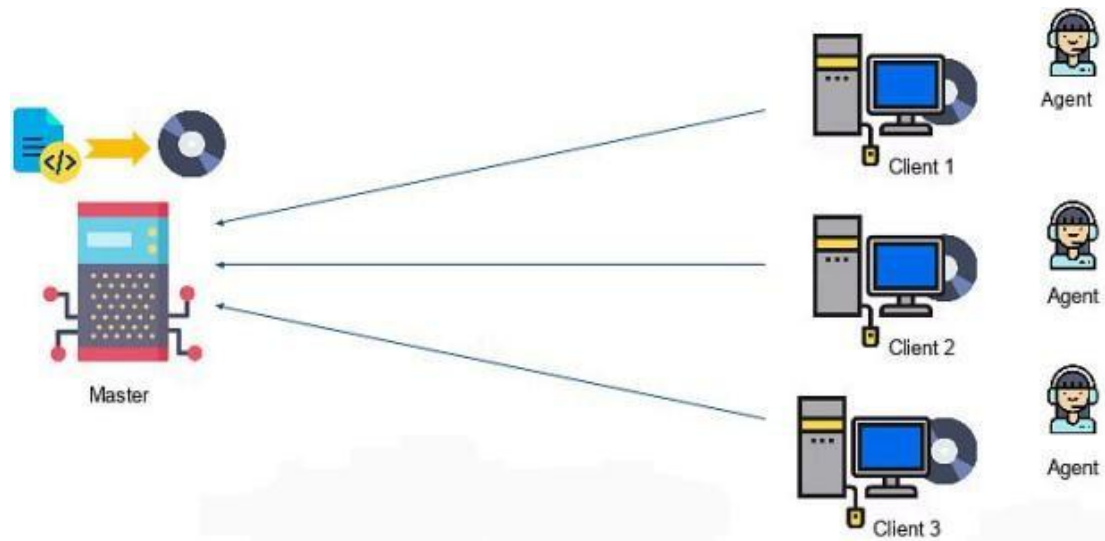
4. What Puppet Can Do

- Install, configure, and manage software packages
- Manage users, groups, and permissions
- Configure and maintain services (e.g., Apache, Nginx, MySQL)
- Enforce security policies and compliance rules
- Automate infrastructure provisioning
- Manage files, directories, and system resources

5. How Puppet Works

Puppet follows a **client-server model**:

1. **Puppet Master (Server):** Holds the configuration (manifests and modules).
2. **Puppet Agent (Client):** Installed on each node (system) that needs to be managed.
3. **Workflow:**
 - Puppet agent sends system facts (OS, hostname, etc.) to the Puppet master.
 - Puppet master compiles a **catalog** (set of configuration instructions).
 - Puppet agent applies the catalog and enforces the configuration.
 - Reports are sent back to the master.



6. How to Use and Write Puppet Manifest Files

- Puppet manifest files use the **.pp** extension.
- They define resources in a **declarative manner**.

Example Manifest File (Apache Installation):

```
package { 'apache2': ensure=>
  installed,
}
```

```
service {
  'apache2': ensure =>
    running,      enable
    => true,
}
```

```
file {'/var/www/html/index.html': ensure => file,
  content => "<h1>Hello from Puppet!</h1>",
}
```

Steps to Use:

1. Write the manifest file (e.g., apache.pp).
2. Apply the manifest using the command:

```
puppet apply apache.pp
```

3. Puppet ensures Apache is installed, running, and configured as described

Conclusion: Puppet is a powerful automation and configuration management tool that simplifies system administration. By using resources like classes, modules, and manifests, administrators can manage thousands of systems with ease, ensuring consistency, compliance, and reliability.

LO MAP: LO1, LO6

(A) Gondal

Assignment 1

Q1) What is ^{Test} Automation? What are advantages of Automation testing?

Sol

- Test Automation refers to the use of specialised tools and scripts to perform software testing tasks automatically. Rather than manually clicking through an application / checking outputs, automation involves writing test scripts that simulate user interactions and validate system responses.
- Automation Testing is an essential part of modern software development especially in Agile, DevOps and continuous Integration and Delivery (CI/CD) practices.
- It ensures that software behaves as expected after code changes by ~~re-running~~ pre-defined tests automatically.
- Automation is primarily applied in the following areas:
 - (i) Regression Testing
 - Ensuring old features still work after changes
 - (ii) Smoke Testing
 - Basic check after deployment
 - (iii) Load and Performance Testing
 - Simulating thousands of users
 - (iv) Cross Browser Testing
 - Ensuring consistent behaviour on different browsers
- Common Test Automation Tools:
 - (i) Selenium (Web Apps)
 - (ii) JUnit / TestNG (Java)
 - (iii) PyTest / Robust Framework (Python)



Computer Society of India

Education Directorate, National Headquarters
CIT Campus, IV Cross Road, Taramani, Chennai - 600 113

Tel: 044-22542874 / 29501103

CSI ACCREDITED STUDENT

Name : Tanishk Dhope

ID. No: 01615468

Valid Till: 30th June 2025

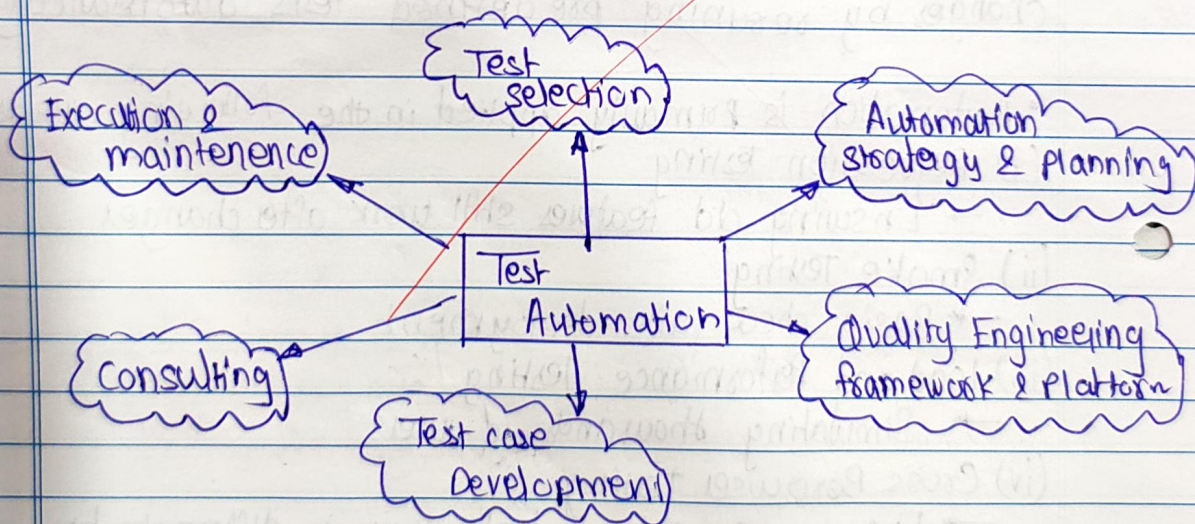
Thadomal Shahani Engineering College, Mumbai - 400050,
Maharashtra.


Authorised Signatory

- (iv) Appium (Mobile Apps)
- (v) JMeter (Performance Testing)
- (vi) Postman / Newman (API testing)

• Advantages of Automation Testing:

- (i) Speed and Efficiency
- (ii) Reusability
- (iii) Reduces Human Error
- (iv) Cost Effective over time
- (v) Supports CI/CD and DevOps
- (vi) Improved Test coverage
- (vii) 24x7 Execution
- (viii) Rapid Feedback to Developers
- (ix) Accuracy in Repeated Tests



Q2 What is XPath? Explain the difference between / and // in XPath.

Soln XPath (XML Path language) is a query language used

to navigate through elements and attributes in XML and HTML documents.

- It is widely used in selenium Web Driver and other automation frameworks to locate web elements on a page.

- In web ~~development~~ automation, elements like buttons, input fields or links must be ~~locate~~ loaded precisely. XPath allows testers to locate such elements based on:

- Their position in the DOM
- Their attributes (like id, class, href etc)
- their text context.

Example: Code in HTML

```
<input type="text" id="username" name="user">
```

we can use XPath to locate this element using:

→ `//*[@id='username']`

- / - Single Slash:

- used when you want to start from the root node
- It goes step by step in the hierarchy.
- Fails if any level is incorrect.

Eg: `/html/body/div[1]/form/input`

→ selects `<input>` inside a specific hierarchy

- // - Double Slash:

- Searches anywhere in the document
- Much more flexible and fault-tolerant

~~test~~ → Doesn't care about exact structure.

Eg: `//input[@id='username']`

→ Selects any input with id='username', even if nested deep.

• When to use '/' or '//':

Situation

Recommended

→ Exact structure is known

/

→ Structure unknown or dynamic

//

→ Testing fixed elements like headers

/html/head/title

→ Finding buttons / inputs regardless of location

//button[text()='Submit']

→ XPath = `//tagname[@Attribute='value']`

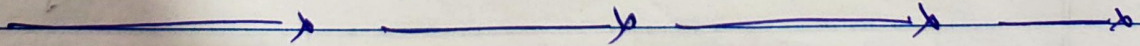
selects
current
node

tagname
like input, div, img

select
Attribute

Attribute
Name

value of
Attribute



DevOps Assignment 2

Q1) What is selenium? what are Selenium Components?
What makes selenium such widely used. Give reasons
why is it advised to select selenium as a testing tool
for web applications or systems.

Soln • Selenium is an open source automated testing framework
used to validate web applications across different
browsers and platforms.
• It allows testers to write test scripts in various
programming languages like Java, Python, C#, Ruby etc.
to automate browser actions like clicking, typing,
navigation or verifying UI elements.

Selenium Components:

- Selenium is not a single tool but a suite of tools each serving a different purpose.

(i) Selenium IDE (Integrated Development Environment)

- A Browser extension (Chrome, Firefox)
- Record and playback tool for quickly creating test scripts.
- Best for beginners or quick prototyping.

(ii) Selenium RC (Remote Control)

- Allowed writing test cases in various languages
- Has been replaced by selenium web driver.

(iii) Selenium WebDriver

- Most widely used component.
- Provides API's to interact directly with browsers.

- Supports multiple browsers
- Nose Flexible.

(iv) Selenium Grid:

- Used for parallel test execution across different machines and browsers
- Saves a lot of time.

Why is selenium widely used?

- Selenium is extremely popular because:

(i) Open Source & Free - No License cost

(ii) Cross Browser support - Works with different brows.

(iii) Cross Platform support - Windows, MacOS etc.

(iv) Multiple language support - Java, Python, C# etc.

(v) Integration with tools - works with Jenkins, Docker etc.

(vi) Parallel execution - Selenium Grid saves a lot of time

(vii) Large Community Support provided by Selenium.

~~(viii)~~
(viii) Scalable and Flexible - Works for small tests & large enterprise systems

Selenium For Web Applications / Systems

- It is advised to select selenium for web testing because:

- Free and open source, reducing project costs.

- Supports all major browsers by ensuring compatibility testing.

- Runs on multiple operating systems, ensures system

wide coverage.

- Supports parallel & distributed execution for faster cycles
- Highly extendable with frameworks like Cucumber, JUnit etc.
- Strong community & documentation to find the easy solutions to the problems

10

15

20

25

30